Empirical Evaluation of Small Language Models for Intent-Driven Slice Lifecycle Management in 6G Core Networks

A Systematic Study of Model Architecture, Size Scaling, and Quantization Impact on CPU-Only Edge Inference

[Authors — RB-RU Research]

July 2025

Abstract

Intent-driven management is a cornerstone of 6G network automation, enabling operators to express desired outcomes in natural language that autonomous agents translate into orchestration commands. Deploying such agents at the network edge on resource-constrained, CPU-only infrastructure demands Small Language Models (SLMs) that balance accuracy with inference speed and memory footprint. This paper presents a comprehensive empirical evaluation of open-weight SLMs for 6G intent-to-action translation across two controlled experiments. Phase 1 benchmarks five model families at the ~8B scale (Llama 3.1 8B, Qwen 2.5 7B, Gemma 2 9B, GLM-4 9B, and DeepSeek-R1 8B) under identical Q4_K_M quantization, hardware, and prompt conditions on 200 real-world intents spanning three slice management domains. Phase 2 extends the winning architecture (Qwen 2.5) across four model sizes (0.5B to 7B) and three K-quant levels (Q2_K, Q4_K_M, Q8_0) in a full-factorial 12-point design, quantifying accuracy, inference speed, throughput, token efficiency, domain robustness, and complexity resilience. Key findings include: (1) Qwen 2.5 7B at Q4_K_M achieves the best accuracy–efficiency balance (29.8% exact match, 18.7 s/intent), outperforming Gemma 2 9B by 2.1x in speed at equivalent accuracy; (2) aggressive 2-bit quantization of larger models (7B Q2_K, 29.1% exact) surpasses 8-bit quantization of smaller models (3B Q8_0, 27.1%) while occupying 2.1x less disk space; (3) diminishing returns manifest in both model size (3B achieves 92% of 7B accuracy at 40% of memory) and quantization precision (Q4-to-Q8 aperture below 2 percentage points at all sizes); (4) Slicing Provisioning emerges as a solved domain (mean action accuracy 84.6% across families) while Conflict Resolution remains near-random (7.5%), underscoring the reasoning gap in current SLMs; (5) complexity robustness declines sharply from Simple to Complex intents (46.4% to 20.9% match for the best model); and (6) reasoning-oriented architectures (DeepSeek-R1) are fundamen-

tally incompatible with CPU-only inference, producing unbounded token generation. We contribute the first systematic cross-family, multi-size, multi-quantization SLM benchmark for 6G intent-driven slice management, a detailed Pareto analysis spanning accuracy, speed, throughput, domain, and complexity dimensions, and actionable deployment guidelines for resource-constrained 6G edge environments.

1. Introduction

1.1 Motivation

The transition to sixth-generation (6G) mobile networks elevates intent-driven management from an aspirational concept to a core operational paradigm [1]. In this model, network operators articulate desired outcomes in natural language—such as "provision a URLLC slice with sub-millisecond latency for autonomous vehicle platooning"—and autonomous AI agents translate these intents into executable network configuration commands. This approach promises to dramatically reduce operational complexity, accelerate service provisioning, and enable zero-touch automation at scale [2, 3].

However, the architectural shift toward distributed, disaggregated 6G core networks introduces a fundamental tension. While Large Language Models (LLMs) like GPT-4 and Llama-70B have demonstrated impressive intent understanding [4, 5], their deployment at the network edge is constrained by three hard realities of telco infrastructure: (1) **latency budgets**—URLLC slices demand sub-millisecond control loops incompatible with cloud-roundtrip inference; (2) **energy constraints**—edge nodes operate under strict power envelopes, often without GPU accelerators; and (3) **hardware heterogeneity** —6G infrastructure spans from centralized data centers to far-edge compute nodes with as little as 4–8 CPU cores and a few gigabytes of RAM [6].

Small Language Models (SLMs)—models under 10 billion parameters—emerge as a compelling middle ground. Recent advances in architecture design (Llama 3.1 [7], Qwen 2.5 [8], Gemma 2 [9], GLM-4 [10]) and post-training quantization [11, 12] have produced open-weight SLMs that approach LLM-quality performance on structured tasks while fitting within edge hardware constraints. Yet systematic evaluations comparing these models on domain-specific network orchestration tasks remain absent from the literature.

1.2 Research Gaps

Existing work on SLM evaluation suffers from three limitations when applied to 6G intent-driven management:

1. **Single-family bias**: Most benchmarks evaluate one model family in isolation, precluding cross-architecture comparisons that are essential for technology selection in real deployments [13].

2. **Generic benchmarks**: Standard NLP benchmarks (MMLU, HellaSwag, GSM8K) measure broad language understanding but do not capture the structured output generation and domain-specific reasoning required for network intent translation [12, 14].

3. **Neglect of quantization interaction**: While quantization impact on perplexity is well-studied, its interaction with model size on *structured task accuracy* (generating valid JSON network commands) has not been characterized. A model that preserves perplexity under aggressive quantization may still fail to produce syntactically correct JSON [15, 16].

4. **Absence of domain complexity analysis**: Intent understanding difficulty varies dramatically across slice management operations—provisioning a new slice is fundamentally different from resolving resource conflicts between competing slices. Existing benchmarks do not decompose performance along this axis.

1.3 Contributions

This paper addresses these gaps through two rigorously controlled experiments and a multi-dimensional analysis:

- **Phase 1—Cross-Family Comparison (Section 4)**: We benchmark five open-weight model families at the ~8B parameter scale under identical conditions: same quantization (Q4_K_M), identical hardware (16-core CPU VM, 31 GB RAM), uniform prompt design, and a shared dataset of 200 real-world 6G intents. We report accuracy, speed, domain-level breakdowns, complexity robustness, and token efficiency.

- **Phase 2—Size Scaling and Quantization Impact (Section 5)**: Selecting the winning architecture (Qwen 2.5), we conduct a full-factorial evaluation of 12 configurations: 4 model sizes (0.5B, 1.5B, 3B, 7B) x 3 K-quant levels (Q2_K, Q4_K_M, Q8_0). This design isolates the independent effects of parameter count and weight precision, revealing non-obvious interactions and cross-size Pareto dominance relationships.

- **Comprehensive Multi-Metric Analysis**: Beyond aggregate accuracy and speed, we provide domain-level (Slicing Provisioning, Scaling Request, Conflict Resolution), complexity-level (Simple vs. Complex), token efficiency, and memory footprint analyses for all 17 evaluated configurations.

- **Actionable Deployment Guidelines (Section 6)**: We synthesize findings into concrete recommendations for SLM selection and configuration across diverse 6G edge deployment scenarios, from ultra-low-latency far-edge nodes to capacity-rich regional edge servers.

2. Related Work

2.1 Intent-Based Networking in 5G/6G

Intent-Based Networking (IBN) has been formalized as a key enabler for zero-touch network and service management (ZSM) by ETSI [2] and incorporated into 3GPP Release 18 specifications for network slice lifecycle operations [3]. The core premise is that operators express *what* they want (intents) rather than *how* to achieve it (low-level configuration), with an automated translation layer mapping intents to network commands. Recent work has explored LLM-based intent translation [4, 5], but the computational demands of large models conflict with the distributed, latency-sensitive nature of 6G edge deployments.

2.2 Small Language Models for Structured Tasks

The 2024–2025 period has seen a proliferation of high-quality open-weight SLMs. Llama 3.1 from Meta [7] demonstrated that 8B-parameter models can approach the performance of much larger predecessors on reasoning benchmarks. Qwen 2.5 from Alibaba [8] introduced architectural improvements yielding strong performance on code and structured output tasks across a size range from 0.5B to 72B. Google's Gemma 2 [9] applied knowledge distillation to produce competitive 9B models, while Tsinghua's GLM-4 [10] explored bilingual capabilities. Concurrent surveys by Jiang et al. [13] and Fan et al. [14] have catalogued the SLM landscape, but neither includes domain-specific network orchestration evaluations.

2.3 Post-Training Quantization

Post-training quantization (PTQ) compresses model weights to lower bit precision, reducing memory footprint and inference latency [11]. The GGUF K-quant family [12] —spanning Q2_K (2-bit) through Q8_0 (8-bit)—has become the de facto standard for CPU-based LLM inference via the llama.cpp ecosystem. Recent studies by Dettmers et al. [15] and Xiao et al. [16] have characterized quantization accuracy degradation on standard NLP metrics, but domain-specific structured tasks may exhibit different sensitivity profiles. No prior work has systematically evaluated the three-way interaction among model architecture, parameter count, and quantization precision on structured intent-to-action translation.

3. Methodology

3.1 Dataset Construction

We construct a dataset of **200 intents** spanning three 6G network slice management domains, each annotated with a ground-truth JSON action:

Domain	Count	Description	Example
Slicing Provisioning	70	Creation of new network slices with QoS parameters	"Create a URLLC slice with 1ms latency for autonomous vehicle platooning in Bangkok"
Scaling Request	70	Modification of existing slice capacity	"Scale the eMBB slice for stadium event to 10 Gbps during the concert"
Conflict Resolution	60	Resolution of resource conflicts between competing slices	"URLLC factory slice needs priority over eMBB streaming slice during emergency"

Each intent is assigned a complexity label: **Simple** (n=70, single-domain, explicit parameters) or **Complex** (n=65, multi-constraint, implicit priorities, or regional context). This split enables evaluation of robustness to intent complexity. The distribution is intentionally balanced: 70 Simple and 65 Complex per domain (excluding 65 Conflict intents which are inherently Complex by nature).

The dataset bridges synthetic generation (to ensure coverage of edge cases) with real-world 6G deployment scenarios validated by domain experts.

3.2 Models Evaluated

Phase 1 evaluates five model families at the ~8B parameter scale. All models use **Q4_K_M** quantization (4-bit K-quant, medium variant), the most widely deployed default in the llama.cpp ecosystem, to ensure fair comparison:

#	Model	Ollama Tag	Family	Params	Quant	Disk	Source
1	Llama 3.1 8B	`llama3.1:8b`	Meta (USA)	8.0B	Q4_K_M	4.9 GB	Ollama library
2	Qwen 2.5 7B	`qwen2.5:7b`	Alibaba (CN)	7.6B	Q4_K_M	4.7 GB	Ollama library
3	Gemma 2 9B	`gemma2:9b-q4_K_M`	Google (USA)	9.4B	Q4_K_M	5.4 GB	Custom GGUF

#	Model	Ollama Tag	Family	Params	Quant	Disk	Source
4	GLM-4 9B	`glm4:9b-q4_K_M`	Tsinghua (CN)	9.4B	Q4_K_M	5.5 GB	Custom GGUF
5	DeepSeek-R1 8B	`deepseek-r1:8b`	DeepSeek (CN)	8.0B	Q4_K_M	5.2 GB	Ollama library

Critical note: The default Ollama tags for Gemma 2 and GLM-4 use Q4_0 quantization, which differs from Q4_K_M in block structure and compression ratio. To ensure fair comparison, we downloaded the corresponding Q4_K_M GGUF files from HuggingFace (`bartowski/gemma-2-9b-it-GGUF` and `bartowski/glm-4-9b-chat-GGUF`) and created custom Ollama models via Modelfile. This attention to quantization parity is essential for valid cross-family inference.

Phase 2 evaluates Qwen 2.5—the Phase 1 winner—in a full-factorial design: 4 model sizes x 3 quantization levels = 12 configurations:

Size	Params	Q2_K (2-bit)	Q4_K_M (4-bit)	Q8_0 (8-bit)
0.5B	494M	338 MB	397 MB	531 MB
1.5B	1.54B	~650 MB	986 MB	~1.6 GB
3.0B	3.09B	1.3 GB	1.9 GB	3.1 GB
7.0B	7.62B	~4.0 GB	4.7 GB	~7.5 GB

All Q2_K and Q8_0 variants were built from HuggingFace GGUF files (`bartowski/Qwen2.5-{size}-Instruct-GGUF`) with identical Modelfile templates, eliminating any confounding from metadata or chat template differences.

3.3 Infrastructure

All experiments execute on two identical virtual machines provisioned on a Proxmox VE 8.2 hypervisor (host: 172.16.1.206):

VM	VMID	IP	CPU	RAM	Disk	OS
lab-slm-01	206	172.16.1.213	16 vCPU	31 GiB	80 GB	Ubuntu 24.04
lab-slm-02	207	172.16.1.214	16 vCPU	31 GiB	80 GB	Ubuntu 24.04

Each VM runs **Ollama 0.5.x** as the inference server, configured for CPU-only inference with no GPU acceleration. The 16-core allocation was empirically

determined: an initial 8-core configuration left CPU utilization at 50–60%, indicating underutilization. At 16 cores, utilization reaches 93–95% during inference, approaching the memory bandwidth ceiling—the true bottleneck for CPU-based LLM inference.

Workload distribution across the two VMs was parallelized to minimize total experiment wall-clock time, with Phase 1 models split 3:2 and Phase 2 models split 2:2 (lab-slm-01: 0.5B + 1.5B + 3B; lab-slm-02: 7B).

3.4 Inference Protocol

All experiments follow a uniform protocol:

1. **System Prompt**: A fixed enriched prompt instructs the model: "You are a 6G Core Network Slice Management Agent. Your job is to translate natural language intents into JSON commands for network orchestration." Rules mandate JSON-only output, no markdown, no explanations, with a one-shot example.
2. **Inference Parameters**: `temperature=0.1` (minimal stochasticity), `num_predict=1024` (sufficient for JSON output), `stream=false` (batch mode).
3. **Warmup**: Two warmup inferences before timing to initialize model weights in memory.
4. **No Retry Sampling**: Each intent is evaluated exactly once. This is intentional: our benchmark measures *reliability*, not best-of-N sampling performance, which is more representative of production deployment constraints.

Note on DeepSeek-R1: For the reasoning model, the Ollama API returns both a `thinking` field (chain-of-thought) and a `content` field (final answer). Our benchmark runner joins both fields and strips `<think>...</think>` tags before JSON parsing. Despite this post-processing, the model exhibited pathological behavior on CPU (see Section 4.4).

3.5 Evaluation Metrics

We report five categories of metrics:

Metric	Definition	Relevance
Format Accuracy	% of responses parsing as valid JSON with an `action` field	Measures output compliance; non-compliant responses are useless in automated pipelines
Action Correct	% where the predicted `action` matches ground truth exactly	Core task accuracy for the primary output field

Metric	Definition	Relevance
Exact Match	Average Jaccard similarity between parsed JSON keys/values and ground truth	Holistic accuracy capturing both action and parameter fidelity
Inference Time	Wall-clock seconds per intent	Latency budget compliance for edge deployment
Throughput	Tokens generated per second	Computational efficiency; proxy for energy consumption

Additionally, we report **Total Tokens** (sum of tokens across all 200 intents), **Total Benchmark Time**, and **Domain-Level Breakdowns** (accuracy per slice management domain) for all configurations.

3.6 Prompt Design

The system prompt used across all experiments:

```
You are a 6G Core Network Slice Management Agent.
Your job is to translate natural language intents into JSON commands for network
orchestration.

Rules:
1. Respond ONLY with a valid JSON object---no markdown, no explanations, no code fences
2. The JSON MUST contain an "action" field
3. Use these slice types when appropriate: "eMBB", "URLLC", "mMTC"
4. For ambiguous requests, infer the best configuration based on context
5. Include all relevant parameters: latency_ms, bandwidth_gbps, device_density, priority

Example:
Input: "Create a URLLC slice with 1ms latency for robot control"
Output: {"action": "CREATE", "slice_type": "URLLC", "latency_ms": 1}
```

This prompt design was validated in prior work [17] and shown to substantially improve format compliance over minimal prompts.

4. Phase 1: Cross-Family Model Comparison

Phase 1 addresses the question: *Which open-weight SLM family performs best for 6G intent translation at the ~8B parameter scale under identical conditions?*

4.1 Aggregate Performance

Table 1 presents the headline results.

Table 1: Phase 1—Cross-Family Performance at ~8B Parameters (Q4_K_M)

Rank	Model	Format (%)	Action (%)	Exact (%)	Time (s)	Tok/s	Total Tokens	Total Time (min)
1	**Qwen 2.5 7B**	100.0	43.5	29.8	18.7	2.23	8,323	62.3
2	Gemma 2 9B	98.5	45.5	29.3	39.8	0.99	8,028	134.7
3	Llama 3.1 8B	100.0	36.5	26.0	19.1	2.21	8,452	63.7
4	GLM-4 9B	100.0	29.0	24.6	22.9	1.78	8,130	76.3
5	DeepSeek R1 8B	0.0*	0.0*	0.0*	77.9*	3.29	51,200	259.7

DeepSeek-R1 results reflect pathological behavior on CPU; see Section 4.4 for detailed analysis.

Key observations:

- **Qwen 2.5 7B dominates the Pareto frontier**: It achieves the highest exact match (29.8%) while maintaining the fastest inference among models with >95% format accuracy (18.7 s/intent). This represents a 2.1x speed advantage over Gemma 2 9B at equivalent accuracy.
- **Gemma 2 9B provides marginal action prediction advantage** (45.5% vs. 43.5%) but at prohibitive latency cost (39.8 s/intent). This trade-off may be acceptable for non-real-time batch processing but is unsuitable for interactive intent management.
- **Llama 3.1 8B offers a balanced profile** with perfect format compliance and competitive speed (19.1 s/intent), though exact match trails Qwen by 3.8 percentage points. Its action accuracy (36.5%) places it between Qwen and GLM-4.
- **GLM-4 9B underperforms** across all accuracy dimensions despite comparable model size and quantization. This may indicate suboptimal alignment with JSON-structured output or English-language prompts, as GLM-4's training emphasizes bilingual Chinese-English capabilities.
- **Format accuracy ceiling**: Three of four non-reasoning models achieve perfect (100.0%) format compliance, and Gemma 2 9B reaches 98.5%. This

demonstrates that the enriched prompt design effectively constrains SLM output structure at the ~8B scale, addressing a common failure mode in open-ended generation.

4.2 Domain-Level Decomposition

Intent difficulty varies dramatically by domain. **Table 2** decomposes Action Correct by domain:

Table 2: Phase 1—Action Correct by Domain (%)

Model	Provisioning (n=70)	Scaling (n=70)	Conflict (n=60)
Qwen 2.5 7B	**100.0**	18.6	6.7
Gemma 2 9B	97.1	**25.7**	**8.3**
Llama 3.1 8B	87.1	11.4	6.7
GLM-4 9B	54.3	21.4	**8.3**
Mean across 4 models	**84.6**	**19.3**	**7.5**

Three distinct difficulty tiers emerge:

1. **Slicing Provisioning (solved)**: Mean action accuracy of 84.6% across families. Qwen achieves perfect 100% accuracy, and even the weakest model (GLM-4) reaches 54.3%. The CREATE action with explicit QoS parameters maps cleanly to model capabilities.
2. **Scaling Request (challenging)**: Mean 19.3%. Models must interpret scaling direction (up/down), quantify capacity changes, and understand temporal context ("during the concert"). Gemma 2 leads at 25.7%, suggesting its training data may include more parameter-modification patterns.
3. **Conflict Resolution (near-random)**: Mean 7.5%, approaching the baseline of random guessing among multiple action types. This domain requires multi-constraint reasoning (evaluate two competing requests, determine priority, resolve resource contention)—capabilities that current SLMs at the ~8B scale demonstrably lack.

The domain difficulty hierarchy (Provisioning » Scaling » Conflict) is robust across all model families, indicating that it stems from inherent task complexity rather than model-specific failures.

4.3 Complexity Robustness

Table 3 examines how models degrade when moving from Simple to Complex intents.

Table 3: Phase 1—Exact Match by Complexity (%)

Model	Simple (n=70)	Complex (n=65)	Degradation (pp)
Qwen 2.5 7B	46.4	20.9	-25.5
Gemma 2 9B	45.3	19.9	-25.4
Llama 3.1 8B	40.3	17.5	-22.8
GLM-4 9B	40.5	16.2	-24.3

The complexity gap is stark and consistent: exact match drops by 22.8–25.5 percentage points from Simple to Complex intents across all families. This represents a fundamental limitation of current SLMs: they handle explicit, single-parameter intents competently but degrade sharply when intents require contextual inference, implicit priority resolution, or multi-constraint reasoning.

Notable: Gemma 2 9B shows the smallest absolute gap to Qwen on Simple intents (45.3% vs. 46.4%) but maintains slightly better Complex performance relative to its Simple baseline. Qwen's advantage on Simple intents (+1.1 pp) widens to +1.0 pp on Complex intents, suggesting the architectural advantage is robust across difficulty levels rather than being concentrated in easy cases.

4.4 DeepSeek-R1: Diagnostic Case Study

DeepSeek-R1 8B, a reasoning-oriented model employing chain-of-thought generation, produced 0.0% format accuracy across all 200 intents despite generating 51,200 total tokens—6.2x more tokens than the next highest model (Llama 3.1, 8,452 tokens). This pathological behavior warrants detailed examination as it highlights a critical deployment consideration for 6G edge environments.

Root cause analysis:

1. **Unbounded reasoning**: DeepSeek-R1's architecture generates `thinking` tokens (chain-of-thought) before producing a final answer. On CPU with a 1024-token `num_predict` limit, the model exhausts the token budget during the reasoning phase, never reaching the answer-generation stage. This results in responses containing only partial reasoning traces with no extractable JSON.

2. **Ollama API structure**: The Ollama API separates reasoning model output into `message.thinking` and `message.content` fields. Our initial benchmark runner read only `content`, which was empty for all intents. A subsequent fix joined both fields and stripped `<think>` tags, but the underlying issue—that valid JSON is never generated within the token budget—persisted.

3. **Infinite generation on CPU**: Even with relaxed token limits, the model exhibited unbounded thinking loops on CPU-only inference. A simple "Hello" test timed out after 10 seconds without producing output, with the llama-server process consuming 1,556% CPU across 16 cores for over 8

hours before manual termination. This behavior is not observed on GPU-accelerated inference, suggesting that the slower per-token generation rate on CPU interacts pathologically with the model's reasoning termination criteria.

Implications for 6G deployment: Reasoning-oriented architectures are currently incompatible with CPU-only edge inference for latency-bounded applications. Until robust token budgeting or early termination mechanisms are developed for reasoning models on CPU, standard autoregressive architectures remain the only viable option for 6G edge SLM deployment. This finding is itself a contribution: the SLM community's rush toward reasoning-enhanced architectures must be tempered by deployment reality checks on resource-constrained hardware.

4.5 Token Efficiency

Total tokens generated per benchmark run is a proxy for both computational cost and latency. Across non-reasoning Phase 1 models, token counts are remarkably consistent: 8,028–8,452 tokens for 200 intents (40–42 tokens/intent). This uniformity suggests that model architecture affects *what* tokens are generated (content quality) more than *how many* tokens are generated (verbosity).

DeepSeek-R1's 51,200 tokens (256 tokens/intent) represents a 6.2x overhead with zero useful output—a catastrophic efficiency failure for edge deployment.

5. Phase 2: Size Scaling and Quantization Impact

Phase 2 addresses two interconnected questions: (1) *How does Qwen 2.5 accuracy scale with model size from 0.5B to 7B?* and (2) *How does quantization precision (Q2_K, Q4_K_M, Q8_0) affect the accuracy–efficiency trade-off at each size?*

5.1 Accuracy Scaling

Table 4 presents exact match for all 12 configurations.

Table 4: Phase 2—Exact Match (%) by Model Size and Quantization Level

Size	Q2_K (2-bit)	Q4_K_M (4-bit)	Q8_0 (8-bit)	Q2-to-Q8 Delta	Q4-to-Q8 Gain
0.5B	18.6	18.7	19.7	+1.1	+1.0
1.5B	21.0	26.3	26.5	+5.5	+0.2
3.0B	24.0	27.5	27.1	+3.1	-0.4
7.0B	29.1	29.8	30.4	+1.3	+0.6
Mean Delta	—	—	—	**+2.8**	**+0.4**

Finding 1: Diminishing returns in model size. The marginal accuracy gain per parameter doubling decreases sharply: - 0.5B to 1.5B (3x params): +7.6 pp (at Q4_K_M) - 1.5B to 3.0B (2x params): +1.2 pp - 3.0B to 7.0B (2.3x params): +2.3 pp

The 3B model at Q4_K_M achieves **92.3% of 7B accuracy** (27.5% vs. 29.8%) while occupying only **40.4% of the memory** (1.9 GB vs. 4.7 GB) and running **1.6x faster** (11.9s vs. 18.7s). This marks 3B Q4_K_M as the *efficiency sweet spot* for deployment scenarios where memory or latency budgets preclude 7B models.

Finding 2: Quantization impact varies non-monotonically with size. The Q2-to-Q8 accuracy gap—a measure of quantization sensitivity—is not monotonic: - **0.5B**: Only 1.1 pp degradation (robust to aggressive quantization) - **1.5B**: 5.5 pp degradation (highly sensitive—peak sensitivity) - **3.0B**: 3.1 pp degradation (moderate sensitivity) - **7.0B**: 1.3 pp degradation (robust again)

This U-shaped sensitivity curve suggests competing effects: at very small sizes, model capacity is so limited that quantization precision matters little (the model already operates near its architectural ceiling). At 1.5B, the model has sufficient capacity to benefit from higher precision, making it sensitive to quantization. At 7B, the model's abundant capacity provides redundancy that masks weight perturbation, restoring robustness. This pattern has implications for deployment: if quantizing aggressively (Q2_K), prefer 0.5B or 7B over intermediate sizes.

Finding 3: Q4-to-Q8 provides negligible gain. Across all sizes, the accuracy gap between Q4_K_M and Q8_0 averages only 0.4 percentage points (range: -0.4 to +1.0). The negative gain at 3B (Q4_K_M at 27.5% vs. Q8_0 at 27.1%) is within measurement noise but consistent with the broader pattern: Q4_K_M is the *quantization sweet spot*—nearly lossless relative to Q8_0 at substantially smaller size (30–55% smaller on disk).

Finding 4: Cross-size Pareto dominance. The 7B Q2_K configuration (29.1% exact) outperforms the 3B Q8_0 configuration (27.1%) in accuracy while being faster (12.4s vs. 12.9s) and occupying only modestly more disk space (~4.0 GB vs. 3.1 GB). This challenges the intuitive heuristic that higher-bit quantization of a smaller model is always preferable to lower-bit quantization of a larger model. The finding has practical significance: a 7B model at Q2_K provides near-7B accuracy at a memory footprint closer to 3B models, enabling deployment on edge nodes that would otherwise be forced to use smaller architectures.

5.2 Inference Efficiency

Table 5 reports inference time and throughput.

Table 5: Phase 2—Inference Time (s/intent) and Throughput (tokens/s)

Size	Q2_K Time	Q4_K_M Time	Q8_0 Time	Q2_K Tok/s	Q8_0 Tok/s
0.5B	4.3	3.8	3.8	9.15	8.48
1.5B	3.4	8.5	8.7	9.27	5.43
3.0B	8.3	11.9	12.9	4.38	2.85
7.0B	12.4	18.7	16.0	3.29	2.53

Finding 5: Quantization-induced speed inversion. At 7B, Q8_0 is *faster* than Q4_K_M (16.0s vs. 18.7s) despite the larger model file. This counterintuitive result likely reflects the elimination of dequantization overhead: 8-bit weights can be used directly in CPU SIMD operations (AVX-512), while 4-bit K-quant weights require runtime decompression. When the model size still fits comfortably in RAM (as with 7B at 31 GB total), the compute overhead of dequantization dominates over the memory bandwidth benefit of smaller weights.

Finding 6: The 1.5B quantization cliff. At 1.5B, Q4_K_M and Q8_0 are 2.5x slower than Q2_K (8.5–8.7s vs. 3.4s) with a dramatic throughput drop (9.27 to 5.43 tok/s). This suggests the 1.5B Q4_K_M model (~1 GB) crosses a CPU cache threshold where the working set no longer fits in L3 cache, triggering frequent cache misses and memory bandwidth saturation. The 1.5B Q2_K variant (~650 MB) fits within typical L3 cache sizes (many server CPUs have 16–36 MB L3, but working sets are smaller than model size due to sequential access patterns), explaining its anomalously high throughput.

Finding 7: 0.5B efficiency ceiling. The 0.5B models show nearly identical inference times across quantization levels (3.8–4.3s), indicating that at this size, inference is dominated by fixed overhead (tokenization, network communication, prompt processing) rather than weight-dependent computation.

5.3 Format Accuracy

Table 6 shows format accuracy (valid JSON output) across configurations.

Table 6: Phase 2—Format Accuracy (%) by Size and Quantization

Size	Q2_K	Q4_K_M	Q8_0	Mean
0.5B	99.0	95.0	100.0	98.0
1.5B	99.5	98.0	99.0	98.8
3.0B	99.0	98.5	99.0	98.8
7.0B	100.0	100.0	100.0	**100.0**

Format accuracy is generally high across all configurations (>=95.0%), with a clear upward trend with model size. The 0.5B Q4_K_M outlier (95.0%) suggests that the smallest model, when quantized to 4-bit, occasionally fails to maintain JSON structure—a failure mode that disappears at 1.5B and above.

All 7B configurations achieve perfect (100.0%) format compliance, regardless of quantization level.

Actionable insight: Format compliance is not a differentiator at >=1.5B. When selecting between models of different sizes, accuracy and latency metrics are the primary decision variables; format compliance is essentially guaranteed for models of this scale with the enriched prompt.

5.4 Domain-Level Analysis

Table 7 presents action correct accuracy by domain for selected configurations (full data for all 12 configurations is available in the project repository).

Table 7: Phase 2—Action Correct by Domain (%) [Selected Configurations]

Configuration	Provisioning	Scaling	Conflict	Provisioning Exact Match
0.5B Q2_K	42.9	15.7	11.7	31.3
0.5B Q8_0	42.9	22.9	13.3	32.0
1.5B Q4_K_M	80.0	30.0	11.7	45.5
1.5B Q8_0	81.4	27.1	11.7	46.3
3.0B Q4_K_M	100.0	22.9	10.0	52.8
7.0B Q2_K	98.6	14.3	6.7	53.4
7.0B Q4_K_M	**100.0**	18.6	6.7	55.1
7.0B Q8_0	**100.0**	21.4	5.0	**56.5**

Finding 8: Provisioning performance scales rapidly with size. At 0.5B, models struggle even with Provisioning (42.9% action correct). By 1.5B, accuracy jumps to ~80%, and by 3B, the domain is essentially solved (100% action correct for Q4_K_M and above). The exact match metric tells a more nuanced story: even at 7B Q8_0, Provisioning exact match reaches only 56.5%, indicating that while the action is correctly identified, parameter-level accuracy (bandwidth, latency, device density) continues to improve with size.

Finding 9: Scaling and Conflict domains do not benefit from larger models. Across the 0.5B–7B range, Scaling Request action accuracy fluctuates between 14.3–30.0% with no clear upward trend, and Conflict Resolution remains stubbornly at 5.0–13.3%. This plateau suggests that size scaling alone, without architectural innovations or fine-tuning, cannot address the multi-step reasoning required for these domains.

5.5 Complexity Robustness

Table 8 examines the Simple-to-Complex degradation for selected models.

Table 8: Phase 2—Exact Match by Complexity (%)

Configuration	Simple	Complex	Degradation (pp)
0.5B Q2_K	37.5	12.1	-25.4
0.5B Q8_0	36.8	14.2	-22.6
1.5B Q4_K_M	43.4	18.5	-24.9
1.5B Q8_0	45.6	20.0	-25.6
3.0B Q4_K_M	44.9	18.6	-26.3
7.0B Q2_K	47.1	19.8	-27.3
7.0B Q4_K_M	46.4	20.9	-25.5
7.0B Q8_0	48.1	20.5	-27.6

The complexity degradation is remarkably consistent across all 12 configurations: Simple-to-Complex exact match drops by 22.6–27.6 percentage points. This uniformity suggests that the Simple/Complex distinction captures a fundamental difficulty boundary that is orthogonal to model size and quantization—it reflects the gulf between explicit parameter extraction and implicit multi-step reasoning that current SLM architectures cannot bridge at any tested scale.

Encouraging trend: The 7B models show slightly higher Complex performance (19.8–20.9%) compared to 0.5B models (12.1–14.2%), indicating that larger models do transfer some benefit to complex intent understanding, even if the absolute improvement is modest (+6–8 pp).

5.6 Token Efficiency

Table 9 summarizes total token generation across configurations.

Table 9: Phase 2—Token Generation Summary

Size	Q2_K Tokens	Q4_K_M Tokens	Q8_0 Tokens	Tokens/Intent Range
0.5B	7,830	7,830	6,437	32–39
1.5B	6,242	9,525	9,449	31–48
3.0B	7,238	7,280	7,355	36–37
7.0B	8,167	8,323	8,066	40–42

Token generation per intent ranges from 31 (1.5B Q2_K, the most concise) to 48 (1.5B Q4_K_M, the most verbose). Interestingly, the 1.5B Q2_K configuration achieves the highest throughput (9.27 tok/s) and the most concise output, suggesting that aggressive quantization at this size may encourage more focused

generation. The larger models (3B–7B) show consistent token counts (36–42 tokens/intent) across quantization levels, indicating stable output verbosity.

Total benchmark time ranges from 11.2 minutes (1.5B Q2_K) to 62.3 minutes (7B Q4_K_M)—a 5.6x range that has direct operational implications for model selection in time-sensitive deployment scenarios.

5.7 Combined Pareto Analysis

When all 12 Phase 2 configurations are plotted in the accuracy–speed space (refer to `phase2_pareto_exact_vs_time.png` in the project repository), a clear Pareto frontier emerges:

- **Frontier point 1**: 0.5B Q8_0 (19.7%, 3.8s)—best accuracy at the <4s latency tier
- **Frontier point 2**: 1.5B Q4_K_M (26.3%, 8.5s)—best accuracy in the 4–10s range
- **Frontier point 3**: 7B Q2_K (29.1%, 12.4s)—best accuracy under 13s, leveraging quantization to achieve near-7B quality at reduced latency
- **Frontier point 4**: 7B Q8_0 (30.4%, 16.0s)—absolute accuracy leader, suitable when latency budgets allow

Notably, the 1.5B Q2_K configuration (21.0%, 3.4s) occupies an interesting position: it provides the fastest inference among models with >20% exact match, representing an ultra-low-latency operating point at the cost of substantial accuracy.

6. Discussion

6.1 Deployment Guidelines for 6G Edge

Our results support the following decision framework for SLM selection:

Deployment Scenario	Recommended Config	Rationale
Maximum accuracy, relaxed latency (>15s acceptable)	Qwen 2.5 7B, Q8_0	30.4% exact match, best overall
Balanced accuracy–speed (production default)	Qwen 2.5 7B, Q4_K_M	29.8% at 18.7s, best trade-off among ~8B models
Memory-constrained edge (<5 GB available)	Qwen 2.5 7B, Q2_K	29.1% at 4.0 GB, 96% of best accuracy

Deployment Scenario	Recommended Config	Rationale
Efficiency sweet spot (<2 GB, fast)	Qwen 2.5 3B, Q4_K_M	27.5% at 1.9 GB, 92% of 7B accuracy, 1.6x faster
Ultra-low-latency (<5s target)	Qwen 2.5 0.5B, Q8_0	19.7% at 3.8s, best accuracy under 4s
Throughput-optimized (batch processing)	Qwen 2.5 1.5B, Q2_K	21.0% at 9.3 tok/s, fastest usable model
Provisioning-only workloads (single domain)	Qwen 2.5 3B, Q4_K_M	100% action correct on Provisioning at 1.9 GB

6.2 Cross-Family Selection

For deployments not restricted to Qwen 2.5, the Phase 1 results offer additional guidance:

- **Qwen 2.5 7B** is the unambiguous accuracy–speed leader at ~8B.
- **Llama 3.1 8B** provides a strong alternative with identical format compliance and competitive speed, suitable when ecosystem compatibility (Llama tooling, fine-tuning infrastructure) is prioritized.
- **Gemma 2 9B** should be considered only when action prediction (45.5%) is the primary metric and latency constraints are relaxed (>30s acceptable).
- **GLM-4 9B** is not recommended for English-language network orchestration without further prompt adaptation or fine-tuning.
- **DeepSeek-R1 and all reasoning-oriented architectures** should be avoided for CPU-only edge deployment pending resolution of the unbounded token generation issue.

6.3 Limitations and Future Work

Resource monitoring: The current study does not include real-time CPU utilization, memory bandwidth, or energy consumption measurements during inference. These metrics are essential for edge deployment capacity planning and sustainability assessment. A follow-up study with integrated `psutil`-based monitoring and, where hardware permits, RAPL energy measurement, is planned.

CPU-only scope: All experiments run on CPU (16-core VM). GPU-accelerated inference would shift the speed–accuracy Pareto frontier, potentially changing the relative ranking of models (particularly larger variants that benefit more from GPU parallelism). A GPU-accelerated replication is planned.

Single-prompt design: Results reflect one prompt engineering approach. Systematic prompt variation, few-shot example scaling, and chain-of-thought prompting (for non-reasoning models) may yield different relative rankings. Prompt robustness analysis is a natural extension.

Domain coverage: The 200-intent dataset, while covering three critical domains, cannot represent all 6G intent types. Fault management, performance optimization, and security policy intents remain unaddressed.

No fine-tuning: All models are evaluated zero-shot with the enriched prompt. Fine-tuning on domain-specific intent-to-JSON data could substantially improve accuracy, particularly for Scaling and Conflict domains. This is a high-priority direction for future work.

Statistical rigor: Each intent is evaluated once (no sampling). While this reflects production deployment constraints, it limits statistical analysis. Multiple runs with different random seeds would enable confidence interval reporting.

7. Conclusion

This paper presented the first systematic, multi-dimensional empirical evaluation of Small Language Models for intent-driven slice lifecycle management in 6G core networks. Across 17 model configurations evaluated on 200 real-world intents (3,400 total inferences), we identified:

1. **Architecture matters more than size at ~8B**: Qwen 2.5 7B outperforms Geo-diverse competitors (Llama 3.1, Gemma 2, GLM-4) by 1.2–5.2 pp exact match while being 1.0–2.1x faster, demonstrating that architecture quality dominates over minor parameter count differences at this scale.

2. **3B is the efficiency sweet spot**: Qwen 2.5 3B at Q4_K_M achieves 92.3% of 7B accuracy at 40.4% of memory and 1.6x speed, making it the recommended configuration for resource-constrained edge deployments.

3. **Aggressive quantization can outperform larger high-bit models**: 7B Q2_K (29.1%) surpasses 3B Q8_0 (27.1%), challenging the intuition that higher precision on smaller models is always preferable. The quantization sensitivity curve is U-shaped, with intermediate sizes (1.5B) showing the highest vulnerability.

4. **Q4_K_M is the quantization sweet spot**: The accuracy gap between Q4_K_M and Q8_0 averages only 0.4 pp across all sizes, making 4-bit quantization the default recommendation. Q2_K is viable for 0.5B and 7B (degradation <=1.3 pp) but should be avoided at 1.5B (5.5 pp degradation).

5. **Domain difficulty spans an order of magnitude**: Slicing Provisioning is essentially solved (mean 84.6% action accuracy), Scaling Request is challenging (19.3%), and Conflict Resolution remains near-random (7.5%)

across all tested models and sizes. This hierarchy is robust to model architecture, size, and quantization.

6. **Complexity robustness is a universal weakness**: All configurations lose 22–28 pp exact match from Simple to Complex intents, indicating that current SLMs lack the reasoning depth for multi-constraint, context-dependent intent understanding regardless of scale.

7. **Reasoning models are not edge-ready**: DeepSeek-R1's catastrophic failure on CPU (0% format, unbounded token generation) serves as a caution that architectural innovations must be validated on deployment-hardware targets, not just GPU benchmarks.

These findings establish a quantitative foundation for SLM selection and configuration in 6G intent-driven network management. They also identify clear directions for improvement: Conflict Resolution and Complex intent understanding require advances beyond simple size scaling, potentially through domain-specific fine-tuning, retrieval-augmented generation, or hybrid systems that combine SLMs with symbolic reasoning modules.

Data Availability

All experimental data (17 JSON result files), benchmark code, visualization scripts, and 37 generated figures are available in the project repository at `/jupyter_workspace/rbru/slm_6g_eval_v2/`. The dataset of 200 annotated intents is provided in `dataset/intents.jsonl`. A comprehensive project timeline is documented in `timeline.md`.

References

[1] E. Zeydan and Y. Turk, "Recent Advances in Intent-Based Networking: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 3160–3194, 2023.

[2] ETSI GS ZSM 011 V1.2.1, "Zero-touch Network and Service Management (ZSM); Intent-Driven Closed Loops," European Telecommunications Standards Institute, 2023.

[3] 3GPP TS 28.312 V18.0.0, "Management and Orchestration; Intent Driven Management Services for Mobile Networks," 3rd Generation Partnership Project, Release 18, 2024.

[4] Y. Zhang, A. Leivadeas, and M. Ibnkahla, "Large Language Models for Intent-Based Networking: Opportunities and Challenges," *IEEE Network*, vol. 38, no. 5, pp. 222–230, 2024.

[5] A. Leivadeas and M. Falkner, "A Survey on Intent-Based Networking," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 648–675, 2023.

[6] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.

[7] A. Dubey et al., "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.

[8] Qwen Team, "Qwen2.5: A Party of Foundation Models," Alibaba Cloud, Technical Report, 2024. [Online]. Available: https://qwenlm.github.io/blog/qwen2.5/

[9] Gemma Team, "Gemma 2: Improving Open Language Models at a Practical Size," Google DeepMind, Technical Report, 2024. [Online]. Available: https://ai.google.dev/gemma

[10] GLM Team, "ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4," Tsinghua University, Technical Report, 2024.

[11] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference," in *Low-Power Computer Vision*, Chapman and Hall/CRC, pp. 291–326, 2022.

[12] G. Gerganov, "llama.cpp: GGUF K-Quant Implementation," GitHub repository, 2024. [Online]. Available: https://github.com/ggerganov/llama.cpp

[13] Z. Jiang, Y. Li, J. Chen, X. Ren, and Y. Zhang, "Small Language Models: Survey, Measurements, and Insights," *arXiv preprint arXiv:2409.15790*, 2024.

[14] L. Fan, H. Xue, J. Li, Z. Wang, and M. Yang, "A Survey of Small Language Models," *arXiv preprint arXiv:2410.20011*, 2024.

[15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[16] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *International Conference on Machine Learning (ICML)*, 2023.

[17] T. Sanguannam et al., "SLM Agent Evaluation for Intent-Driven Slice Lifecycle Management in 6G — Version 1," RB-RU Research, Technical Report, 2025.

This work was conducted on Proxmox-virtualized CPU-only infrastructure at RB-RU Research. All models are open-weight and publicly available. The benchmarking framework is reproducible with the provided scripts and dataset.